UATM GASA FORMATION
Université Africaine de Technologie et de Management

RAPPORT DE PROJET

Conception et développement d'une application de gestion de stock

Application StockPro

FILIÈRE
Système Informatique et Logiciel (SIL2)

ANNÉE ACADÉMIQUE
2025 – 2026

SOMMAIRE

INTRODUCTION GÉNÉRALE **3**
Problématique 3
Hypothèses 3
Choix du sujet 3
1. Explication et fonctionnalités de l'application **4**
1.1. Gestion des produits 4
1.2. Gestion des catégories 4
1.3. Gestion des entrées 4
1.4. Gestion des sorties 4
1.5. Alertes de stock faible 4
1.6. Authentification 4
1.7. Consultation de l'état du stock 4
1.8. Inventaire du stock 4
1.9. Statistiques globales 4
1.10. Gestion des accès 4
2. Analyse **5**
2.1. Diagramme de cas d'utilisation 5
2.2. Description des acteurs 5
2.3. Description des cas d'utilisation 6
2.4. Relations entre les cas d'utilisation 6
3. Conception **7**
3.1. Diagramme de classes 7
3.2. Description des classes 8
3.3. Relations et règles de gestion 9
4. Modèle logique de données relationnel (MLDR) **10**
4.1. Règles de passage du diagramme de classes au MLDR 10
4.2. Modèle logique de données relationnel 10
4.3. Récapitulatif des clés étrangères 11
4.4. Dictionnaire de données 11
4.5. Index pour accélérer la recherche 14
5. Implémentation de la base de données **15**
5.1. Création de la base 15
5.2. Fonctionnement automatique 15
5.3. Exécution et vérification 15
6. Réalisation de l'application **16**
6.1. Environnement de développement 16
6.2. Interfaces de l'application 16
7. Extraits de code **20**
7.1. Authentification 20
7.2. Gestion des produits 20
7.3. Enregistrement des entrées et des sorties 20
7.4. Alertes de stock faible 21
7.5. Inventaire du stock 21
CONCLUSION GÉNÉRALE **22**
ANNEXE A — SCRIPT SQL COMPLET **23**

INTRODUCTION GÉNÉRALE

La gestion des stocks constitue une activité importante dans le suivi des produits et des marchandises. Elle permet de connaître les quantités disponibles, de suivre les mouvements de produits et d'éviter les situations de rupture ou de surplus. Cependant, lorsque le suivi est effectué manuellement, il peut devenir difficile de garder une information fiable et à jour sur l'état du stock.

L'évolution des outils informatiques offre aujourd'hui la possibilité de simplifier cette gestion grâce à des applications dédiées. Ces solutions permettent de centraliser les informations, d'enregistrer les entrées et les sorties de produits et de suivre plus facilement les quantités disponibles. Elles peuvent également intégrer des alertes afin de signaler lorsqu'un produit atteint un niveau de stock faible.

C'est dans cette perspective que s'inscrit notre projet intitulé « Conception et développement d'une application de gestion de stock ». L'application proposée permettra principalement de gérer les produits et leurs catégories, d'enregistrer les entrées et les sorties et de générer des alertes lorsque le stock d'un produit devient insuffisant.

Problématique

Le suivi manuel des stocks peut entraîner des erreurs dans les quantités enregistrées et rendre difficile l'identification rapide des produits en faible quantité. Il devient alors nécessaire de disposer d'un outil permettant de faciliter et d'améliorer ce suivi.

La problématique de ce projet est donc la suivante :

> ***Comment concevoir une application permettant d'assurer un suivi efficace des stocks, de gérer les mouvements des produits et de prévenir les situations de stock faible ?***

Hypothèses

Pour répondre à cette problématique, nous retenons les hypothèses suivantes :

- **H1 :** La mise en place d'une application informatisée permettrait d'améliorer le suivi des stocks.
- **H2 :** L'enregistrement des entrées et des sorties permettrait de connaître plus précisément les quantités disponibles.
- **H3 :** La mise en place d'un système d'alerte permettrait de détecter rapidement les produits dont le stock est faible.

Choix du sujet

Le choix de ce sujet s'explique par l'importance d'un suivi régulier et fiable des stocks. La conception de cette application permettra de mettre en place une solution simple pour faciliter la gestion des produits, suivre leurs mouvements et limiter les problèmes liés au manque d'information sur les quantités disponibles.

Ce projet nous permettra également de mettre en pratique nos connaissances en analyse, conception et développement d'une application informatique.

1. Explication et fonctionnalités de l'application

L'application de gestion de stock est une solution informatique conçue pour faciliter le suivi des produits et des mouvements de stock. Elle permet de centraliser les informations relatives aux produits et de connaître à tout moment les quantités disponibles.

L'application repose principalement sur les fonctionnalités suivantes :

1.1. Gestion des produits

Cette fonctionnalité permet d'ajouter, de modifier, de supprimer et de consulter les produits enregistrés. Chaque produit peut être associé à une catégorie et à une quantité disponible.

1.2. Gestion des catégories

Elle permet de créer et d'organiser les différentes catégories auxquelles les produits appartiennent. Cette organisation facilite la recherche et le classement des produits.

1.3. Gestion des entrées

Cette fonctionnalité permet d'enregistrer l'ajout de produits dans le stock. Chaque entrée entraîne une mise à jour automatique de la quantité disponible.

1.4. Gestion des sorties

Elle permet d'enregistrer les produits retirés du stock. La quantité disponible est automatiquement diminuée après chaque sortie.

1.5. Alertes de stock faible

L'application permet de détecter les produits dont la quantité atteint un seuil défini. Une alerte est alors affichée afin de signaler qu'un réapprovisionnement peut être nécessaire.

1.6. Authentification

Elle permet aux utilisateurs de se connecter afin d'accéder à l'application. Chaque utilisateur doit être authentifié avant d'utiliser les fonctionnalités qui lui sont accessibles.

1.7. Consultation de l'état du stock

Elle permet de consulter à tout moment les quantités disponibles de chaque produit. C'est à partir de cette consultation que peuvent être signalés les produits dont le stock est faible.

1.8. Inventaire du stock

Elle permet de vérifier les quantités réellement présentes dans le stock et de faire le point sur l'ensemble des produits.

1.9. Statistiques globales

Elle permet de suivre l'évolution du stock de manière synthétique grâce à des statistiques globales.

1.10. Gestion des accès

Elle permet à l'administrateur de gérer les droits d'accès des utilisateurs à l'application.

Ces différentes fonctionnalités permettent ainsi d'assurer un suivi plus simple et plus précis des stocks, tout en facilitant la consultation des informations relatives aux produits.

2. Analyse

2.1. Diagramme de cas d'utilisation

Définition : le diagramme de cas d'utilisation permet de décrire les besoins des utilisateurs (acteurs) vis-à-vis du système. Il présente les différentes fonctionnalités auxquelles chaque acteur peut accéder et montre les interactions entre les utilisateurs et l'application.

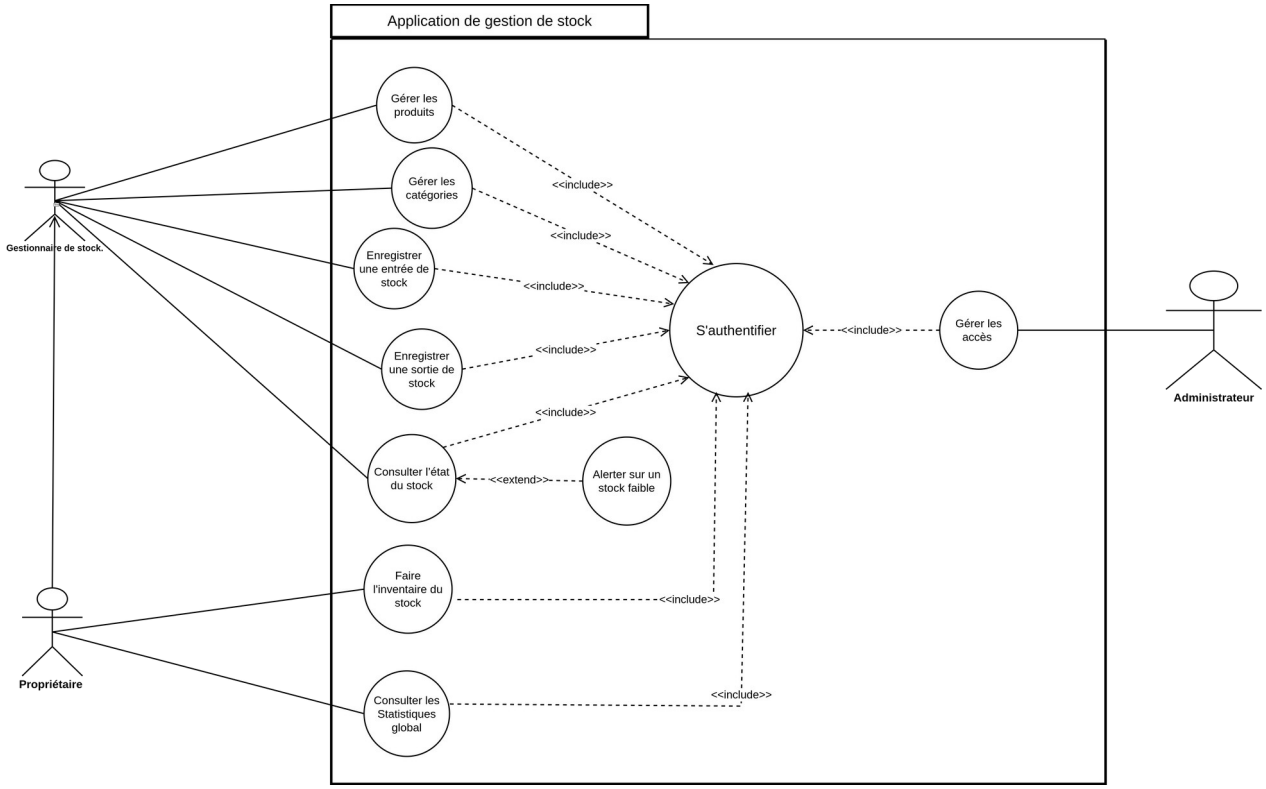


Figure 1 : Diagramme de cas d'utilisation de l'application de gestion de stock

2.2. Description des acteurs

Le diagramme fait intervenir trois acteurs, chacun disposant de fonctionnalités propres :

Tableau 1 : Acteurs du système

Acteur	Rôle	Cas d'utilisation accessibles
Gestionnaire de stock	Utilisateur chargé de la gestion courante du stock.	Gérer les produits Gérer les catégories Enregistrer une entrée de stock Enregistrer une sortie de stock Consulter l'état du stock
Propriétaire	Responsable du suivi global du stock.	Faire l'inventaire du stock Consulter les statistiques globales
Administrateur	Responsable de la gestion des droits d'accès à l'application.	Gérer les accès

Le lien orienté du Propriétaire vers le Gestionnaire de stock représente une relation d'héritage entre acteurs : le Propriétaire peut accéder à toutes les fonctionnalités du Gestionnaire de stock, en plus de ses propres fonctionnalités.

2.3. Description des cas d'utilisation

Le tableau ci-dessous décrit l'ensemble des cas d'utilisation représentés dans le diagramme :

Tableau 2 : Description des cas d'utilisation

Cas d'utilisation	Acteur	Description
S'authentifier	Tous les acteurs	Vérifier l'identité de l'utilisateur avant tout accès aux fonctionnalités de l'application.
Gérer les produits	Gestionnaire de stock	Ajouter, modifier, supprimer et consulter les produits enregistrés.
Gérer les catégories	Gestionnaire de stock	Créer et organiser les catégories auxquelles les produits appartiennent.
Enregistrer une entrée de stock	Gestionnaire de stock	Enregistrer l'ajout de produits dans le stock ; la quantité disponible est mise à jour automatiquement.
Enregistrer une sortie de stock	Gestionnaire de stock	Enregistrer les produits retirés du stock ; la quantité disponible est automatiquement diminuée.
Consulter l'état du stock	Gestionnaire de stock	Consulter les quantités disponibles des produits.
Alerter sur un stock faible	Système	Signaler qu'un produit a atteint le seuil défini afin d'indiquer qu'un réapprovisionnement peut être nécessaire.
Faire l'inventaire du stock	Propriétaire	Effectuer l'inventaire des produits présents dans le stock.
Consulter les statistiques globales	Propriétaire	Consulter les statistiques globales relatives au stock.
Gérer les accès	Administrateur	Gérer les accès des utilisateurs à l'application.

2.4. Relations entre les cas d'utilisation

Le diagramme met en évidence deux types de relations entre les cas d'utilisation :

- **Relation « include » :** tous les cas d'utilisation (gérer les produits, gérer les catégories, enregistrer une entrée, enregistrer une sortie, consulter l'état du stock, faire l'inventaire, consulter les statistiques globales et gérer les accès) incluent le cas « S'authentifier ». L'utilisateur doit donc être authentifié avant d'accéder à ces fonctionnalités.
- **Relation « extend » :** le cas « Alerter sur un stock faible » étend le cas « Consulter l'état du stock ». L'alerte ne se déclenche que lorsque la quantité d'un produit atteint le seuil défini.

3. Conception

3.1. Diagramme de classes

Définition : le diagramme de classes représente la structure statique du système : les classes, leurs attributs et leurs méthodes, ainsi que les relations qui les relient.

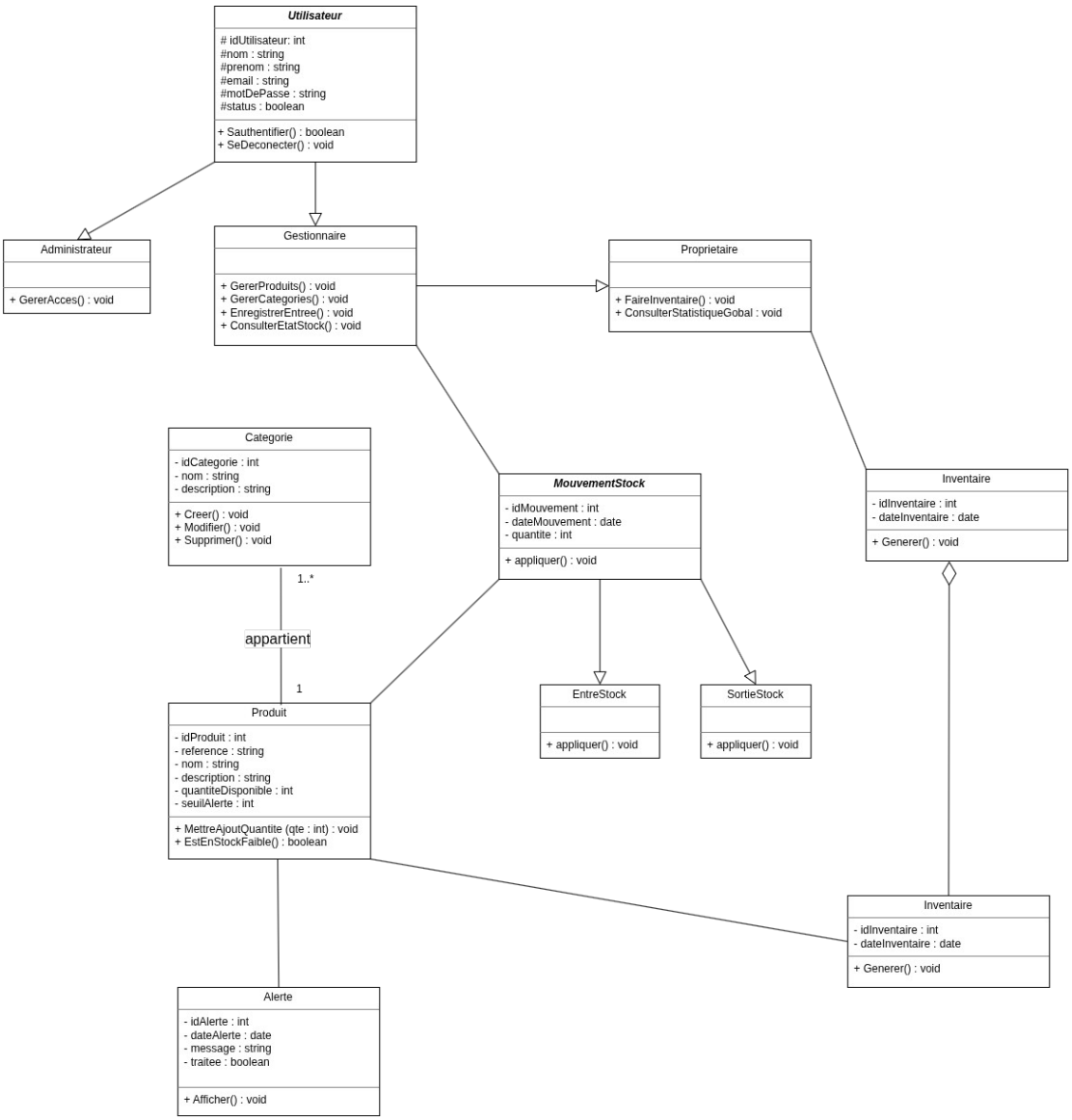


Figure 2 : Diagramme de classes de l'application de gestion de stock

3.2. Description des classes

Le tableau ci-dessous présente les douze classes retenues et leur rôle dans l'application :

Tableau 3 : Description des classes

Classe	Attributs principaux	Rôle
Utilisateur (abstraite)	idUtilisateur, nom, prenom, login, motDePasse, actif	Représente toute personne utilisant l'application ; assure l'authentification.
Gestionnaire	Hérite de Utilisateur	Gère les produits et les catégories, enregistre les entrées et les sorties et consulte l'état du stock.
Propriétaire	Hérite de Gestionnaire	Réalise l'inventaire et consulte les statistiques globales.
Administrateur	Hérite de Utilisateur	Gère les accès des utilisateurs à l'application.
Catégorie	idCategorie, nom, description	Regroupe les produits pour faciliter leur recherche et leur classement.
Produit	idProduit, reference, nom, description, quantiteDisponible, seuilAlerte	Représente un produit suivi en stock ; indique si son stock est faible.
MouvementStock (abstraite)	idMouvement, dateMouvement, quantite	Regroupe ce qui est commun aux entrées et aux sorties de stock.
EntréeStock	Hérite de MouvementStock	Enregistre l'ajout de produits ; augmente la quantité disponible.
SortieStock	Hérite de MouvementStock	Enregistre le retrait de produits ; diminue la quantité disponible.
Alerte	idAlerte, dateAlerte, message, traitee	Signale qu'un produit a atteint le seuil défini.
Inventaire	idInventaire, dateInventaire	Représente une opération d'inventaire du stock.
LigneInventaire	quantiteTheorique, quantiteReelle	Compare, pour un produit, la quantité enregistrée et la quantité constatée.

3.3. Relations et règles de gestion

Les relations entre les classes sont les suivantes :

Tableau 4 : Relations entre les classes

Relation	Multiplicités	Signification
Catégorie – Produit	1 – 0..*	Une catégorie contient plusieurs produits ; un produit appartient à une seule catégorie.
Produit – MouvementStock	1 – 0..*	Un produit peut faire l'objet de plusieurs mouvements (entrées et sorties).
Gestionnaire – MouvementStock	1 – 0..*	Un gestionnaire enregistre plusieurs mouvements.
Produit – Alerte	1 – 0..*	Un produit peut déclencher plusieurs alertes de stock faible.
Propriétaire – Inventaire	1 – 0..*	Le propriétaire réalise plusieurs inventaires.
Inventaire – LigneInventaire	1 – 1..* (composition)	Un inventaire est composé d'au moins une ligne ; une ligne n'existe pas sans son inventaire.
Produit – LigneInventaire	1 – 0..*	Un produit peut apparaître dans plusieurs lignes d'inventaire.

Trois relations d'héritage structurent le modèle : Gestionnaire et Administrateur héritent de Utilisateur, Propriétaire hérite de Gestionnaire (comme dans le diagramme de cas d'utilisation), et EntréeStock et SortieStock héritent de MouvementStock.

Le diagramme traduit également les règles de gestion suivantes :

- Chaque entrée augmente automatiquement la quantité disponible du produit concerné.
- Chaque sortie diminue automatiquement la quantité disponible du produit concerné.
- Une alerte est générée lorsque la quantité disponible d'un produit atteint son seuil d'alerte.
- Les statistiques globales ne sont pas stockées dans une classe : elles sont calculées à partir des produits et des mouvements de stock.

4. Modèle logique de données relationnel (MLDR)

4.1. Règles de passage du diagramme de classes au MLDR

Le MLDR est obtenu à partir du diagramme de classes en appliquant les règles suivantes :

Tableau 5 : Règles de passage du diagramme de classes au MLDR

Élément UML	Règle appliquée	Résultat
Classes persistantes	Chaque classe devient une relation (table) avec une clé primaire	8 tables
Héritage Utilisateur → Administrateur, Gestionnaire, Superviseur	Table unique avec une colonne discriminante	UTILISATEUR.role_utilisateur
Héritage MouvementStock → EntreStock, SortieStock	Table unique avec une colonne discriminante	MOUVEMENT_STOCK.type_mouvement
Association 1 — 0..*	La clé primaire du côté 1 migre en clé étrangère du côté 0..*	9 clés étrangères
Agrégation Inventaire — LigneInventaire (1..*)	Clé étrangère dans LIGNE_INVENTAIRE avec suppression en cascade	ON DELETE CASCADE
Attribut dérivé « ecart »	Colonne calculée (quantité réelle – quantité théorique)	GENERATED ALWAYS … STORED
Méthodes appliquer(), EstEnStockFaible()	Mise à jour automatique du stock et génération d'alerte	Triggers et vue SQL

Remarque : le Superviseur hérite du Gestionnaire. Il est donc stocké avec le rôle SUPERVISEUR et l'application lui accorde aussi les droits du gestionnaire. Dans le MLDR, le rôle « Propriétaire » de l'analyse est désigné par le terme « Superviseur ».

4.2. Modèle logique de données relationnel

Légende : clé primaire soulignée en gras ; clé étrangère précédée de # et en italique.

UTILISATEUR (**id_utilisateur**, nom, prenom, email, mot_de_passe, statut, role_utilisateur)

FOURNISSEUR (**id_fournisseur**, raison_sociale, code_fournisseur, personne_contact, telephone, email, ville, *#id_gestionnaire*)

CATEGORIE (**id_categorie**, nom, *#id_gestionnaire*)

PRODUIT (**id_produit**, reference, nom, description, quantite_disponible, seuil_alerte, prix, *#id_categorie*, *#id_gestionnaire*)

MOUVEMENT_STOCK (**id_mouvement**, date_mouvement, quantite, type_mouvement, *#id_produit*, *#id_gestionnaire*)

INVENTAIRE (**id_inventaire**, date_inventaire, *#id_superviseur*)

LIGNE_INVENTAIRE (**id_ligne_inventaire**, quantite_theorique, quantite_reelle, ecart, *#id_inventaire*, *#id_produit*)

ALERTE (**id_alerte**, date_alerte, message, traitee, *#id_produit*)

4.3. Récapitulatif des clés étrangères

Tableau 6 : Clés étrangères du modèle

Table	Clé étrangère	Référence	Association UML	ON DELETE
fournisseur	id_gestionnaire	utilisateur (id_utilisateur)	gérer	RESTRICT
categorie	id_gestionnaire	utilisateur (id_utilisateur)	créer	RESTRICT
produit	id_categorie	categorie (id_categorie)	appartient	RESTRICT
produit	id_gestionnaire	utilisateur (id_utilisateur)	enregistre	RESTRICT
mouvement_stock	id_produit	produit (id_produit)	concerne	RESTRICT
mouvement_stock	id_gestionnaire	utilisateur (id_utilisateur)	effectuer	RESTRICT
inventaire	id_superviseur	utilisateur (id_utilisateur)	réalise	RESTRICT
ligne_inventaire	id_inventaire	inventaire (id_inventaire)	comprend	CASCADE
ligne_inventaire	id_produit	produit (id_produit)	recense	RESTRICT
alerte	id_produit	produit (id_produit)	génère	CASCADE

Toutes les clés étrangères utilisent ON UPDATE CASCADE.

4.4. Dictionnaire de données

Tableau 7 : Table utilisateur

Colonne	Type	Contraintes	Description
id_utilisateur	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant de l'utilisateur
nom	VARCHAR(100)	NOT NULL	Nom
prenom	VARCHAR(100)	NOT NULL	Prénom
email	VARCHAR(150)	NOT NULL, UNIQUE	Adresse e-mail (identifiant de connexion)
mot_de_passe	VARCHAR(255)	NOT NULL	Mot de passe haché
statut	TINYINT(1)	NOT NULL, DEFAULT 1, CHECK (0 ou 1)	Compte actif (1) ou inactif (0)
role_utilisateur	ENUM	NOT NULL, DEFAULT 'GESTIONNAIRE'	ADMINISTRATEUR, GESTIONNAIRE ou SUPERVISEUR

Tableau 8 : Table fournisseur

Colonne	Type	Contraintes	Description
id_fournisseur	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant du fournisseur
raison_sociale	VARCHAR(150)	NOT NULL	Raison sociale
code_fournisseur	VARCHAR(30)	NOT NULL, UNIQUE	Code du fournisseur
personne_contact	VARCHAR(100)	NULL	Personne à contacter
telephone	VARCHAR(30)	NULL	Téléphone
email	VARCHAR(150)	NULL	Adresse e-mail
ville	VARCHAR(100)	NULL	Ville
id_gestionnaire	INT UNSIGNED	FK, NOT NULL	Gestionnaire qui gère le fournisseur

Tableau 9 : Table categorie

Colonne	Type	Contraintes	Description
id_categorie	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant de la catégorie
nom	VARCHAR(100)	NOT NULL, UNIQUE	Nom de la catégorie
id_gestionnaire	INT UNSIGNED	FK, NOT NULL	Gestionnaire qui crée la catégorie

Tableau 10 : Table produit

Colonne	Type	Contraintes	Description
id_produit	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant du produit
reference	VARCHAR(50)	NOT NULL, UNIQUE	Référence du produit
nom	VARCHAR(150)	NOT NULL	Nom du produit
description	TEXT	NULL	Description
quantite_disponible	INT	NOT NULL, DEFAULT 0, CHECK (>= 0)	Quantité en stock
seuil_alerte	INT	NOT NULL, DEFAULT 0, CHECK (>= 0)	Seuil de déclenchement de l'alerte
prix	DECIMAL(12,2)	NOT NULL, DEFAULT 0, CHECK (>= 0)	Prix unitaire
id_categorie	INT UNSIGNED	FK, NOT NULL	Catégorie du produit
id_gestionnaire	INT UNSIGNED	FK, NOT NULL	Gestionnaire qui enregistre le produit

Tableau 11 : Table mouvement_stock

Colonne	Type	Contraintes	Description
id_mouvement	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant du mouvement
date_mouvement	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date du mouvement
quantite	INT	NOT NULL, CHECK (> 0)	Quantité déplacée
type_mouvement	ENUM	NOT NULL	ENTREE ou SORTIE
id_produit	INT UNSIGNED	FK, NOT NULL	Produit concerné
id_gestionnaire	INT UNSIGNED	FK, NOT NULL	Gestionnaire qui effectue le mouvement

Tableau 12 : Table inventaire

Colonne	Type	Contraintes	Description
id_inventaire	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant de l'inventaire
date_inventaire	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date de l'inventaire
id_superviseur	INT UNSIGNED	FK, NOT NULL	Superviseur qui réalise l'inventaire

Tableau 13 : Table ligne_inventaire

Colonne	Type	Contraintes	Description
id_ligne_inventaire	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant de la ligne
quantite_theorique	INT	NOT NULL, CHECK (>= 0)	Quantité attendue en stock
quantite_reelle	INT	NOT NULL, CHECK (>= 0)	Quantité comptée
ecart	INT	GENERATED ALWAYS (STORED)	quantite_reelle - quantite_theorique
id_inventaire	INT UNSIGNED	FK, NOT NULL, UNIQUE avec id_produit	Inventaire contenant la ligne
id_produit	INT UNSIGNED	FK, NOT NULL	Produit recensé

Tableau 14 : Table alerte

Colonne	Type	Contraintes	Description
id_alerte	INT UNSIGNED	PK, AUTO_INCREMENT	Identifiant de l'alerte
date_alerte	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Date de l'alerte
message	VARCHAR(255)	NOT NULL	Message affiché
traitee	TINYINT(1)	NOT NULL, DEFAULT 0, CHECK (0 ou 1)	Alerte traitée (1) ou non (0)
id_produit	INT UNSIGNED	FK, NOT NULL	Produit concerné

4.5. Index pour accélérer la recherche

Les clés primaires, les colonnes UNIQUE (email, reference, code_fournisseur, nom de catégorie) et les clés étrangères sont déjà indexées automatiquement par InnoDB. Les index ci-dessous complètent ceux-là pour les recherches les plus fréquentes.

Tableau 15 : Index de recherche

Index	Table (colonnes)	Recherche accélérée
idx_produit_nom	produit (nom)	Recherche d'un produit par début de nom (LIKE 'Cah%') et tri par nom
ft_produit_recherche (FULLTEXT)	produit (nom, description)	Recherche par mot-clé (MATCH … AGAINST)
idx_produit_stock	produit (quantite_disponible, seuil_alerte)	Liste des produits en stock faible
idx_mouvement_produit_date	mouvement_stock (id_produit, date_mouvement)	Historique des mouvements d'un produit trié par date
idx_mouvement_date	mouvement_stock (date_mouvement)	Mouvements sur une période
idx_alerte_produit_traitee	alerte (id_produit, traitee)	Alertes non traitées d'un produit (utilisé par le trigger)
idx_alerte_traitee_date	alerte (traitee, date_alerte)	Liste des alertes à traiter, les plus récentes d'abord
idx_fournisseur_nom	fournisseur (raison_sociale)	Recherche d'un fournisseur par nom
idx_fournisseur_ville	fournisseur (ville)	Filtre des fournisseurs par ville
idx_inventaire_date	inventaire (date_inventaire)	Recherche et tri des inventaires par date

Exemple de recherche par mot-clé :

```
SELECT * FROM produit WHERE MATCH(nom, description) AGAINST('cahier');
```

5. Implémentation de la base de données

5.1. Création de la base

Le script SQL a été testé sur MariaDB 10.11 (compatible MySQL 8.0+). Il comprend la création de la base, des 8 tables, des contraintes, de 10 index de recherche, d'une vue, de 3 triggers et d'un jeu de données de test. Le script complet figure en Annexe A.

5.2. Fonctionnement automatique

Les triggers et la vue définis dans la base assurent les comportements suivants sans intervention de l'utilisateur :

- Une ENTREE ou une SORTIE dans mouvement_stock met à jour quantite_disponible du produit.
- Une SORTIE supérieure au stock disponible est refusée (erreur « Stock insuffisant »).
- Quand le stock passe sous le seuil d'alerte, une alerte est créée (une seule alerte non traitée par produit).
- Seul un utilisateur de rôle SUPERVISEUR peut réaliser un inventaire.

5.3. Exécution et vérification

La capture ci-dessous présente la base de données après l'exécution du script.



Figure 3 : Base de données gestion_stock après exécution du script

6. Réalisation de l'application

6.1. Environnement de développement

Le tableau suivant présente les technologies utilisées pour la réalisation de l'application :

Tableau 16 : Technologies utilisées

Domaine	Technologie
Langage de programmation	
Framework / bibliothèques	
Interface web	HTML, CSS, Tailwind CSS
Système de gestion de base de données	MySQL 8.0+ / MariaDB (moteur InnoDB)
Outils de développement	

6.2. Interfaces de l'application

Cette section présente les principales interfaces de l'application, regroupées par acteur.

Page de connexion

Avant tout accès, l'utilisateur doit s'authentifier afin d'utiliser les fonctionnalités qui lui sont accessibles.

Insérer la capture d'écran ici

Figure 4 : Page de connexion

Gestion des produits et des catégories

Le gestionnaire de stock ajoute, modifie, supprime et consulte les produits, et organise les catégories.

Insérer la capture d'écran ici

Figure 5 : Gestion des produits et des catégories

Enregistrement des entrées et des sorties

Chaque entrée ou sortie met automatiquement à jour la quantité disponible du produit concerné.

Insérer la capture d'écran ici

Figure 6 : Enregistrement des entrées et des sorties

État du stock et alertes de stock faible

Les quantités disponibles sont consultables à tout moment ; une alerte signale les produits ayant atteint leur seuil.

Insérer la capture d'écran ici

Figure 7 : État du stock et alertes de stock faible

Inventaire du stock

Le superviseur compare les quantités théoriques aux quantités réellement présentes dans le stock.

Insérer la capture d'écran ici

Figure 8 : Inventaire du stock

Statistiques globales

Le superviseur suit l'évolution du stock de manière synthétique.

Insérer la capture d'écran ici

Figure 9 : Statistiques globales

Gestion des accès

L'administrateur gère les droits d'accès des utilisateurs à l'application.

Insérer la capture d'écran ici

Figure 10 : Gestion des accès

7. Extraits de code

Cette section présente les extraits de code les plus significatifs de l'application.

7.1. Authentification

Code de connexion et de vérification de l'identité de l'utilisateur.

```
// Collez ici votre code
```

7.2. Gestion des produits

Code d'ajout, de modification, de suppression et de consultation des produits.

```
// Collez ici votre code
```

7.3. Enregistrement des entrées et des sorties

Code d'enregistrement d'un mouvement de stock et de mise à jour de la quantité disponible.

```
// Collez ici votre code
```

7.4. Alertes de stock faible

Code de détection et d'affichage des produits ayant atteint leur seuil d'alerte.

```
// Collez ici votre code
```

7.5. Inventaire du stock

Code de réalisation de l'inventaire et de calcul des écarts.

```
// Collez ici votre code
```

CONCLUSION GÉNÉRALE

Ce projet avait pour objectif de concevoir une application de gestion de stock permettant d'assurer un suivi efficace des produits, de gérer leurs mouvements et de prévenir les situations de stock faible. Pour y répondre, nous avons conçu et développé StockPro, une application qui centralise les informations relatives aux produits et à leurs catégories, enregistre les entrées et les sorties, et signale les produits dont la quantité atteint le seuil défini.

La démarche suivie a permis de passer progressivement du besoin à la solution. L'analyse a identifié trois acteurs — le gestionnaire de stock, le propriétaire (superviseur) et l'administrateur — ainsi que les cas d'utilisation qui leur sont associés, dont l'authentification, commune à toutes les fonctionnalités. La conception a ensuite abouti à un diagramme de classes de douze classes, traduit en un modèle logique de données relationnel de huit tables, puis en une base de données dotée de contraintes d'intégrité, d'index de recherche, d'une vue et de triggers qui automatisent la mise à jour des quantités et la génération des alertes.

Les hypothèses formulées en introduction sont ainsi prises en compte par la solution proposée : l'informatisation du suivi des stocks (H1), l'enregistrement systématique des entrées et des sorties pour connaître avec précision les quantités disponibles (H2) et le système d'alerte permettant de détecter rapidement les produits en stock faible (H3).

Sur le plan personnel, ce travail nous a permis de mettre en pratique nos connaissances en analyse, en modélisation UML, en conception de bases de données et en développement d'applications web. Plusieurs évolutions peuvent être envisagées : l'association entre les fournisseurs et les produits, prévue comme extension optionnelle de la base de données, ainsi que l'enrichissement des statistiques globales et des alertes.

ANNEXE A — SCRIPT SQL COMPLET

A.1. Script principal

Script de création de la base gestion_stock : tables, contraintes, index, vue, triggers, jeu de données de test et requêtes de vérification. Si l'outil utilisé ne gère pas DELIMITER, les triggers sont à exécuter un par un.

```
-- ================================================================================
-- StockPro - Application de gestion de stock
-- Script SQL complet : creation de la base, des tables, contraintes,
-- index, vue, triggers et jeu de donnees de test.
-- SGBD cible : MySQL 8.0+ (compatible MariaDB 10.5+)
-- Moteur : InnoDB - Encodage : utf8mb4
-- ================================================================================
DROP DATABASE IF EXISTS gestion_stock;
CREATE DATABASE gestion_stock
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE gestion_stock;

-- --------------------------------------------------------------------------------
-- 1. TABLE UTILISATEUR
-- Heritage Utilisateur -> Administrateur / Gestionnaire / Superviseur
-- traduit par une table unique + colonne discriminante role_utilisateur
-- --------------------------------------------------------------------------------
CREATE TABLE utilisateur (
    id_utilisateur INT UNSIGNED NOT NULL AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL,
    mot_de_passe VARCHAR(255) NOT NULL,
    statut TINYINT(1) NOT NULL DEFAULT 1,
    role_utilisateur ENUM('ADMINISTRATEUR','GESTIONNAIRE','SUPERVISEUR')
        NOT NULL DEFAULT 'GESTIONNAIRE',
    CONSTRAINT pk_utilisateur PRIMARY KEY (id_utilisateur),
    CONSTRAINT uq_utilisateur_email UNIQUE (email),
    CONSTRAINT ck_utilisateur_statut CHECK (statut IN (0, 1))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- --------------------------------------------------------------------------------
-- 2. TABLE FOURNISSEUR
-- Association : Gestionnaire (1) --- gere --- (0..*) Fournisseur
-- --------------------------------------------------------------------------------
CREATE TABLE fournisseur (
    id_fournisseur INT UNSIGNED NOT NULL AUTO_INCREMENT,
    raison_sociale VARCHAR(150) NOT NULL,
    code_fournisseur VARCHAR(30) NOT NULL,
    personne_contact VARCHAR(100) NULL,
    telephone VARCHAR(30) NULL,
    email VARCHAR(150) NULL,
    ville VARCHAR(100) NULL,
    id_gestionnaire INT UNSIGNED NOT NULL,
    CONSTRAINT pk_fournisseur PRIMARY KEY (id_fournisseur),
    CONSTRAINT uq_fournisseur_code UNIQUE (code_fournisseur),
    CONSTRAINT fk_fournisseur_gestionnaire
        FOREIGN KEY (id_gestionnaire) REFERENCES utilisateur (id_utilisateur)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
-- ------------------------------------------------------------------------
-- 3. TABLE CATEGORIE
-- Association : Gestionnaire (1) --- creer --- (0..*) Categorie
-- ------------------------------------------------------------------------
CREATE TABLE categorie (
    id_categorie INT UNSIGNED NOT NULL AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL,
    id_gestionnaire INT UNSIGNED NOT NULL,
    CONSTRAINT pk_categorie PRIMARY KEY (id_categorie),
    CONSTRAINT uq_categorie_nom UNIQUE (nom),
    CONSTRAINT fk_categorie_gestionnaire
        FOREIGN KEY (id_gestionnaire) REFERENCES utilisateur (id_utilisateur)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 4. TABLE PRODUIT
-- Associations : Categorie (1) --- appartient --- (0..*) Produit
-- Gestionnaire (1) --- enregistre --- (0..*) Produit
-- ------------------------------------------------------------------------
CREATE TABLE produit (
    id_produit INT UNSIGNED NOT NULL AUTO_INCREMENT,
    reference VARCHAR(50) NOT NULL,
    nom VARCHAR(150) NOT NULL,
    description TEXT NULL,
    quantite_disponible INT NOT NULL DEFAULT 0,
    seuil_alerte INT NOT NULL DEFAULT 0,
    prix DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    id_categorie INT UNSIGNED NOT NULL,
    id_gestionnaire INT UNSIGNED NOT NULL,
    CONSTRAINT pk_produit PRIMARY KEY (id_produit),
    CONSTRAINT uq_produit_reference UNIQUE (reference),
    CONSTRAINT ck_produit_quantite CHECK (quantite_disponible >= 0),
    CONSTRAINT ck_produit_seuil CHECK (seuil_alerte >= 0),
    CONSTRAINT ck_produit_prix CHECK (prix >= 0),
    CONSTRAINT fk_produit_categorie
        FOREIGN KEY (id_categorie) REFERENCES categorie (id_categorie)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_produit_gestionnaire
        FOREIGN KEY (id_gestionnaire) REFERENCES utilisateur (id_utilisateur)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 5. TABLE MOUVEMENT_STOCK
-- Heritage MouvementStock -> EntreStock / SortieStock
-- traduit par une table unique + colonne discriminante type_mouvement
-- Associations : Gestionnaire (1) --- effectuer --- (0..*) MouvementStock
-- MouvementStock (0..*) --- concerne --- (1) Produit
-- ------------------------------------------------------------------------
CREATE TABLE mouvement_stock (
    id_mouvement INT UNSIGNED NOT NULL AUTO_INCREMENT,
    date_mouvement DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    quantite INT NOT NULL,
    type_mouvement ENUM('ENTREE','SORTIE') NOT NULL,
    id_produit INT UNSIGNED NOT NULL,
    id_gestionnaire INT UNSIGNED NOT NULL,
```

```
    CONSTRAINT pk_mouvement_stock PRIMARY KEY (id_mouvement),
    CONSTRAINT ck_mouvement_quantite CHECK (quantite > 0),
    CONSTRAINT fk_mouvement_produit
        FOREIGN KEY (id_produit) REFERENCES produit (id_produit)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_mouvement_gestionnaire
        FOREIGN KEY (id_gestionnaire) REFERENCES utilisateur (id_utilisateur)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 6. TABLE INVENTAIRE
-- Association : Superviseur (1) --- realise --- (0..*) Inventaire
-- ------------------------------------------------------------------------
CREATE TABLE inventaire (
    id_inventaire INT UNSIGNED NOT NULL AUTO_INCREMENT,
    date_inventaire DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    id_superviseur INT UNSIGNED NOT NULL,
    CONSTRAINT pk_inventaire PRIMARY KEY (id_inventaire),
    CONSTRAINT fk_inventaire_superviseur
        FOREIGN KEY (id_superviseur) REFERENCES utilisateur (id_utilisateur)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 7. TABLE LIGNE_INVENTAIRE
-- Agregation : Inventaire (1) <>--- comprend ---(1..*) LigneInventaire
-- Association : Produit (1) --- recense --- (0..*) LigneInventaire
-- ecart = quantite_reelle - quantite_theorique (colonne calculee)
-- ------------------------------------------------------------------------
CREATE TABLE ligne_inventaire (
    id_ligne_inventaire INT UNSIGNED NOT NULL AUTO_INCREMENT,
    quantite_theorique INT NOT NULL,
    quantite_reelle INT NOT NULL,
    ecart INT GENERATED ALWAYS AS (quantite_reelle - quantite_theorique) STORED,
    id_inventaire INT UNSIGNED NOT NULL,
    id_produit INT UNSIGNED NOT NULL,
    CONSTRAINT pk_ligne_inventaire PRIMARY KEY (id_ligne_inventaire),
    CONSTRAINT uq_ligne_inv_produit UNIQUE (id_inventaire, id_produit),
    CONSTRAINT ck_ligne_inv_theorique CHECK (quantite_theorique >= 0),
    CONSTRAINT ck_ligne_inv_reelle CHECK (quantite_reelle >= 0),
    CONSTRAINT fk_ligne_inv_inventaire
        FOREIGN KEY (id_inventaire) REFERENCES inventaire (id_inventaire)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_ligne_inv_produit
        FOREIGN KEY (id_produit) REFERENCES produit (id_produit)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 8. TABLE ALERTE
-- Association : Produit (1) --- genere --- (0..*) Alerte
-- ------------------------------------------------------------------------
CREATE TABLE alerte (
    id_alerte INT UNSIGNED NOT NULL AUTO_INCREMENT,
    date_alerte DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    message VARCHAR(255) NOT NULL,
    traitee TINYINT(1) NOT NULL DEFAULT 0,
```

```
    id_produit INT UNSIGNED NOT NULL,
    CONSTRAINT pk_alerte PRIMARY KEY (id_alerte),
    CONSTRAINT ck_alerte_traitee CHECK (traitee IN (0, 1)),
    CONSTRAINT fk_alerte_produit
        FOREIGN KEY (id_produit) REFERENCES produit (id_produit)
        ON UPDATE CASCADE ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- ------------------------------------------------------------------------
-- 9. INDEX COMPLEMENTAIRES (recherche rapide)
-- Les cles primaires, les colonnes UNIQUE (email, reference, code_fournisseur,
-- nom de categorie) et les cles etrangeres sont deja indexees par InnoDB.
-- ------------------------------------------------------------------------
-- Produits : recherche par nom (debut de mot), par mot-cle, et stock faible
CREATE INDEX idx_produit_nom ON produit (nom);
CREATE INDEX idx_produit_stock ON produit (quantite_disponible, seuil_alerte);
CREATE FULLTEXT INDEX ft_produit_recherche ON produit (nom, description);

-- Mouvements : historique d'un produit, filtre par periode
CREATE INDEX idx_mouvement_produit_date ON mouvement_stock (id_produit, date_mouvement);
CREATE INDEX idx_mouvement_date ON mouvement_stock (date_mouvement);

-- Alertes : alertes non traitees d'un produit (trigger) et liste des alertes a traiter
CREATE INDEX idx_alerte_produit_traitee ON alerte (id_produit, traitee);
CREATE INDEX idx_alerte_traitee_date ON alerte (traitee, date_alerte);

-- Fournisseurs et inventaires : recherche et tri
CREATE INDEX idx_fournisseur_nom ON fournisseur (raison_sociale);
CREATE INDEX idx_fournisseur_ville ON fournisseur (ville);
CREATE INDEX idx_inventaire_date ON inventaire (date_inventaire);

-- ------------------------------------------------------------------------
-- 10. VUE : produits en stock faible (methode EstEnStockFaible())
-- ------------------------------------------------------------------------
CREATE OR REPLACE VIEW v_produits_stock_faible AS
SELECT p.id_produit,
       p.reference,
       p.nom,
       p.quantite_disponible,
       p.seuil_alerte,
       c.nom AS categorie
FROM produit p
INNER JOIN categorie c ON c.id_categorie = p.id_categorie
WHERE p.quantite_disponible <= p.seuil_alerte;

-- ------------------------------------------------------------------------
-- 11. TRIGGERS
-- (si votre outil ne gere pas DELIMITER, executez-les un par un)
-- ------------------------------------------------------------------------
DELIMITER $$

-- 11.1 Refuse une sortie superieure au stock disponible
CREATE TRIGGER trg_mouvement_stock_bi
BEFORE INSERT ON mouvement_stock
FOR EACH ROW
BEGIN
    DECLARE v_stock INT;
    IF NEW.type_mouvement = 'SORTIE' THEN
```

```
        SELECT quantite_disponible INTO v_stock
            FROM produit
            WHERE id_produit = NEW.id_produit;
        IF v_stock < NEW.quantite THEN
            SIGNAL SQLSTATE '45000'
                SET MESSAGE_TEXT = 'Stock insuffisant pour effectuer cette sortie';
        END IF;
    END IF;
END$$

-- 11.2 Applique le mouvement au stock (methode appliquer()) et genere
-- une alerte si le seuil est atteint (methode EstEnStockFaible())
CREATE TRIGGER trg_mouvement_stock_ai
AFTER INSERT ON mouvement_stock
FOR EACH ROW
BEGIN
    DECLARE v_stock INT;
    DECLARE v_seuil INT;
    DECLARE v_nom VARCHAR(150);
    IF NEW.type_mouvement = 'ENTREE' THEN
        UPDATE produit
            SET quantite_disponible = quantite_disponible + NEW.quantite
            WHERE id_produit = NEW.id_produit;
    ELSE
        UPDATE produit
            SET quantite_disponible = quantite_disponible - NEW.quantite
            WHERE id_produit = NEW.id_produit;
    END IF;
    SELECT quantite_disponible, seuil_alerte, nom
        INTO v_stock, v_seuil, v_nom
        FROM produit
        WHERE id_produit = NEW.id_produit;
    IF v_stock <= v_seuil THEN
        IF NOT EXISTS (SELECT 1 FROM alerte
            WHERE id_produit = NEW.id_produit
            AND traitee = 0) THEN
            INSERT INTO alerte (message, id_produit)
                VALUES (CONCAT('Stock faible pour le produit ', v_nom,
                ' : ', v_stock, ' unite(s) restante(s)'),
                NEW.id_produit);
        END IF;
    END IF;
END$$

-- 11.3 Seul un utilisateur de role SUPERVISEUR peut realiser un inventaire
CREATE TRIGGER trg_inventaire_bi
BEFORE INSERT ON inventaire
FOR EACH ROW
BEGIN
    DECLARE v_role VARCHAR(20);
    SELECT role_utilisateur INTO v_role
        FROM utilisateur
        WHERE id_utilisateur = NEW.id_superviseur;
    IF v_role IS NULL OR v_role <> 'SUPERVISEUR' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Seul un superviseur peut realiser un inventaire';
    END IF;
END$$
```

```
DELIMITER ;

-- ----------------------------------------------------------------------------
-- 12. JEU DE DONNEES DE TEST
-- (les mots de passe sont des valeurs factices : a remplacer par des
-- hash generes par votre application, ex. bcrypt)
-- ----------------------------------------------------------------------------
INSERT INTO utilisateur (nom, prenom, email, mot_de_passe, statut, role_utilisateur) VALUES
    ('ADMIN', 'Systeme', 'admin@stockpro.local', '$2y$10$REMPLACER.PAR.UN.HASH.BCRYPT.ADMIN', 1,
'ADMINISTRATEUR'),
    ('DOSSOU', 'Marc', 'gestionnaire@stockpro.local', '$2y$10$REMPLACER.PAR.UN.HASH.BCRYPT.GEST', 1,
'GESTIONNAIRE'),
    ('HOUNSOU', 'Awa', 'superviseur@stockpro.local', '$2y$10$REMPLACER.PAR.UN.HASH.BCRYPT.PROP', 1,
'SUPERVISEUR');
INSERT INTO fournisseur (raison_sociale, code_fournisseur, personne_contact, telephone, email, ville,
id_gestionnaire) VALUES
    ('Societe Beninoise de Distribution', 'FOUR-001', 'Paul Agbo', '+22901000001',
'contact@sbd.example', 'Cotonou', 2),
    ('Papeterie Centrale', 'FOUR-002', 'Rose Akpovi', '+22901000002', 'ventes@papcentral.example',
'Porto-Novo', 2);
INSERT INTO categorie (nom, id_gestionnaire) VALUES
    ('Boissons', 2),
    ('Fournitures de bureau', 2);
INSERT INTO produit (reference, nom, description, quantite_disponible, seuil_alerte, prix, id_categorie,
id_gestionnaire) VALUES
    ('PRD-0001', 'Eau minerale 1,5 L', 'Bouteille d eau minerale de 1,5 litre', 0, 10, 500.00, 1, 2),
    ('PRD-0002', 'Cahier 200 pages', 'Cahier grand format 200 pages', 0, 20, 1000.00, 2, 2),
    ('PRD-0003', 'Stylo bleu', 'Stylo a bille bleu', 0, 15, 250.00, 2, 2);

-- Les triggers mettent a jour quantite_disponible automatiquement
INSERT INTO mouvement_stock (quantite, type_mouvement, id_produit, id_gestionnaire) VALUES
    (50, 'ENTREE', 1, 2),
    (100, 'ENTREE', 2, 2),
    (30, 'ENTREE', 3, 2),
    (45, 'SORTIE', 1, 2); -- stock = 5 <= seuil 10 : une alerte est generee
INSERT INTO inventaire (id_superviseur) VALUES (3);
INSERT INTO ligne_inventaire (id_inventaire, id_produit, quantite_theorique, quantite_reelle) VALUES
    (1, 1, 5, 5),
    (1, 2, 100, 98),
    (1, 3, 30, 30);

-- ----------------------------------------------------------------------------
-- 13. REQUETES DE VERIFICATION
-- ----------------------------------------------------------------------------
SELECT id_produit, reference, nom, quantite_disponible, seuil_alerte FROM produit;
SELECT * FROM alerte;
SELECT * FROM v_produits_stock_faible;
SELECT * FROM ligne_inventaire;
```

A.2. Extension optionnelle : lien Fournisseur — Produit

Le diagramme de classes ne relie pas Fournisseur à Produit. Si cette association est ajoutée (un fournisseur fournit 0..* produits), la commande suivante est à exécuter après le script principal :

```
ALTER TABLE produit
    ADD COLUMN id_fournisseur INT UNSIGNED NULL,
    ADD CONSTRAINT fk_produit_fournisseur
        FOREIGN KEY (id_fournisseur) REFERENCES fournisseur (id_fournisseur)
        ON UPDATE CASCADE ON DELETE SET NULL;
```